Secure Cloud

L'Aventure d'une Startup Qui Change le Monde

PROLOGUE : L'IDÉE QUI CHANGE TOUT



Marseille, Quartier de la Joliette, 8h00 du matin

Thomas referme son ordinateur d'un geste décidé dans les bureaux qu'il vient de louer face au port. Depuis trois mois, il travaille sur une idée qui pourrait

révolutionner la cybersécurité : créer la première plateforme de communication vraiment inviolable.

Il vous montre les spécifications qu'il a rédigées :

"SECURECLOUD - Communications Professionnelles Sécurisées"

> *"J'ai le financement de démarrage. J'ai des bureaux avec vue sur la Méditerranée. Il me faut maintenant l'équipe technique capable de concrétiser cette vision. Le marché nous attend : entreprises sensibles, ONG, journalistes d'investigation. Vous voulez relever le défi ?"*

L'aventure technique commence.

Contexte



Les entreprises et les journalistes ont besoin de confidentialité absolue. WhatsApp et Telegram, c'est bien pour les particuliers, mais insuffisant pour les enjeux professionnels critiques. Nous allons donc créer un écosystème complet : messagerie sécurisée, partage de fichiers chiffré, infrastructure auto-déployable

Cahier des charges fonctionnel

Vision Produit

Développer une plateforme de communication sécurisée de niveau professionnel, résistante à toutes les formes d'interception et d'espionnage.

Segments Cibles

- **Journalisme d'investigation** : Protection des sources et documents sensibles
- **ONG internationales** : Communications sécurisées en zones hostiles
- **Entreprises sensibles** : Finance, santé, défense
- **Administrations** : Souveraineté numérique et conformité

Analyse Concurrentielle

Signal

- Chiffrement E2E éprouvé
- Interface peu professionnelle, pas de gestion d'entreprise

Slack Enterprise

- Interface moderne, adoption massive
- Slack peut lire tous les messages, pas de chiffrement E2E

Microsoft Teams

- Intégration entreprise, support 24/7
- ✕Microsoft peut accéder aux données, surveillance gouvernementale

Notre avantage : Sécurité absolue avec expérience utilisateur professionnelle.

SPÉCIFICATIONS FONCTIONNELLES DÉTAILLÉES

L'Incident qui Redéfinit Tout

Jour 3, 14h30, bureaux SecureCloud

Thomas entre dans le bureau, téléphone à la main. Par la baie vitrée, on aperçoit les ferries qui partent vers la Corse.

> *"Appel urgent : Médecins Sans Frontières nous contacte. Ils ont 45 000 volontaires dans 70 pays, certains en zones de conflit où leurs communications sont surveillées. Actuellement, ils utilisent WhatsApp et Gmail pour coordonner leurs missions."*

Il se dirige vers le tableau blanc.

> *"Ils nous donnent trois mois pour livrer une solution complète. Voici leurs besoins critiques :"*

Module 1 : Messagerie Sécurisée

Story Utilisateur Principale

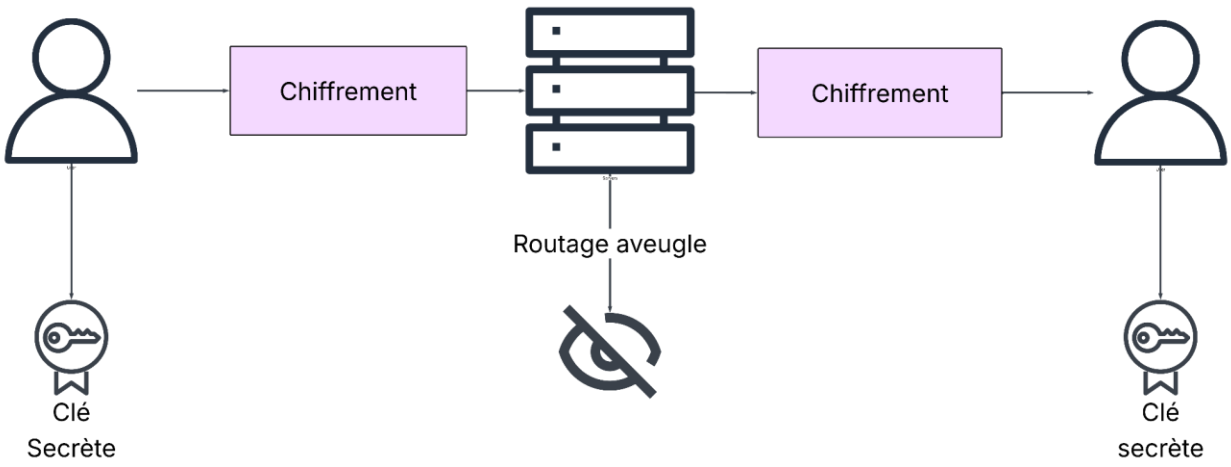
En tant que *coordinateur humanitaire en zone de conflit*
Je veux *communiquer avec mes équipes sans risquer leur sécurité*
Afin de *coordonner des missions humanitaires en toute confidentialité*

Fonctionnalités Core

Messagerie Temps Réel Sécurisé

- Conversations 1-to-1 et groupes jusqu'à 100 participants
- Chiffrement de bout en bout avec algorithme AES-256
- Interface temps réel avec WebSockets sécurisés
- Statuts de présence et notifications intelligentes
- Threading model optimisé pour charge élevée

Architecture technique



"Le serveur ne voit JAMAIS le contenu. Il ne fait que router des paquets chiffrés."

Justification Technique C++

Thomas explique l'architecture :

> *"La messagerie, c'est le cœur du système. 10 000 messages par seconde en pic de charge, temps réel critique, zéro tolérance à la*

panne. Avec C++, on a le contrôle total : gestion mémoire optimisée, threads natifs, performance prédictible."

Gestion Avancée des Conversations

- Recherche dans l'historique (indexation chiffrée locale)
- Threads de conversation pour organisation
- Statuts de présence sécurisés

Module 2 : Partage de fichiers sécurisé

Thomas sort des documents confidentiels...

"MSF manipule des téraoctets de données médicales sensibles. Avec C++, on peut implémenter un système de fichiers chiffré ultra-performant, sans dépendre de solutions cloud externes."

Story Utilisateur Principale

En tant que *responsable médical MSF*
Je veux *partager des dossiers patients avec d'autres centres*
Afin de *coordonner les soins tout en respectant la confidentialité médicale*

Justification C++

"Gestion de fichiers = I/O intensif, compression, chiffrement streaming. C++ nous donne la performance nécessaire pour chiffrer des téraoctets sans ralentir l'interface utilisateur."

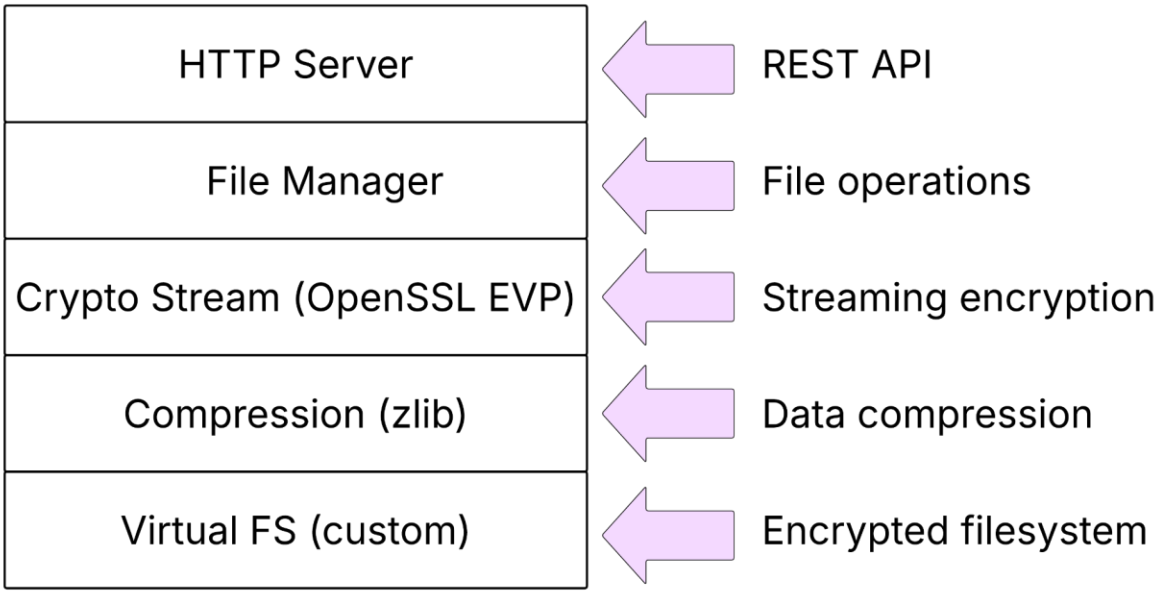
Fonctionnalités Core

Service de Stockage C++ Haute Performance

- Serveur HTTP/HTTPS natif

- Chiffrement streaming avec chunking automatique
- Compression zlib intégrée
- Système de fichiers virtuel chiffré
- Déduplication au niveau blocs

Architecture C++ :



Module 3 Authentification & Autorisation

Thomas pointe le tableau avec gravité :

“La sécurité est un enjeu critique de performance et de fiabilité. Avec C++, nous allons utiliser des bibliothèques cryptographiques éprouvées, légères et auditées, afin de garantir un haut niveau de contrôle tout en évitant les pièges d’implémentations maison”

Il vous montre des statistiques alarmantes sur son écran.

"*Vous savez combien de breaches de sécurité sont dues à des authentifications faibles ? 81% ! Et coûtent en moyenne 4.45 millions de dollars. MSF ne peut pas se permettre une seule faille.*"

Story Utilisateur Principale

En tant qu' *administrateur système MSF*
Je veux *gérer centralement les accès de 45 000 utilisateurs*
Afin de *garantir la sécurité sans impacter les performances*

API REST Centralisée pour Tous les Services

Thomas ouvre l'interface de développement

> *"Notre microservice Auth expose une API REST que tous les autres services consomment. Chaque endpoint est optimisé pour un cas d'usage spécifique."*

Exemples d'Endpoints Principaux du Microservice

POST /auth/login : Authentification initiale avec vérification multi-facteurs

POST /auth/refresh : Renouvellement de token JWT avec validation continue

GET /auth/validate : Validation rapide de token (cache optimisé)

POST /auth/permissions : Vérification des droits pour action spécifique

POST /auth/revoke : Révocation immédiate de tous les tokens utilisateur

Justification C++

Thomas trace un comparatif performance au tableau

"Authentification = calculs cryptographiques intensifs. JWT custom, bcrypt optimisé, gestion mémoire sécurisée... tout ça demande C++ pour la performance."

BENCHMARK AUTHENTIFICATION (1000 logins/seconde)

Node.js bcrypt	VS	C++ bcrypt (OpenSSL)
850ms response 280MB RAM usage 85% CPU usage GC pauses		45ms response 12MB RAM usage 23% CPU usage Zero GC overhead

Fonctionnalités Core

- Gestion des Utilisateurs et Identités
- Système de Permissions Granulaires
- Sécurité et Chiffrement
- API REST pour Microservices

Module 4 : Interface Utilisateur Qt



L'Enjeu Critique de l'Interface

Thomas lance plusieurs applications sur son écran :

"MSF utilise actuellement un mix d'outils : WhatsApp sur mobile, Skype sur desktop, FTP en ligne de commande pour les gros fichiers, emails non chiffrés... C'est l'anarchie totale. Notre interface Qt doit remplacer TOUT ça. Notre interface doit marcher sur un laptop poussiéreux à 45°C avec une connexion satellite de 256kbps en Afghanistan. Electron/Web ne tiendrait pas 10 minutes."

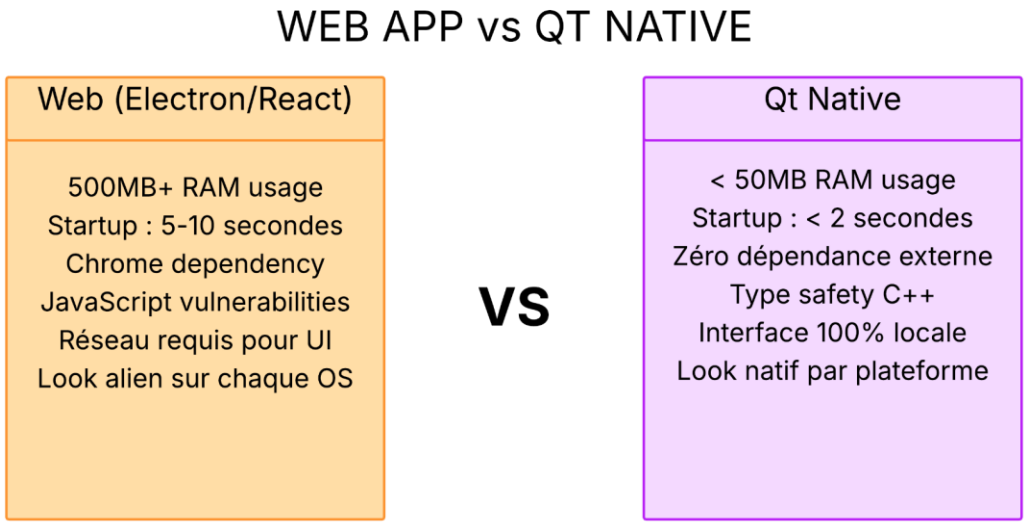
Justification Qt

"Qt nous donne une interface native sur Windows, Linux, macOS. Performance optimale, pas de navigateur intermédiaire, contrôle total sur l'expérience utilisateur."

Thomas ouvre son laptop et vous montre des captures d'écran d'applications Qt professionnelles.

"Regardez Autodesk Maya, VLC Media Player, Adobe Photoshop Elements... Toutes ces applications utilisent Qt. Pourquoi ? Parce que quand vous avez besoin de performance, de stabilité et d'une interface vraiment native, il n'y a pas d'alternative."

Il trace un comparatif au tableau

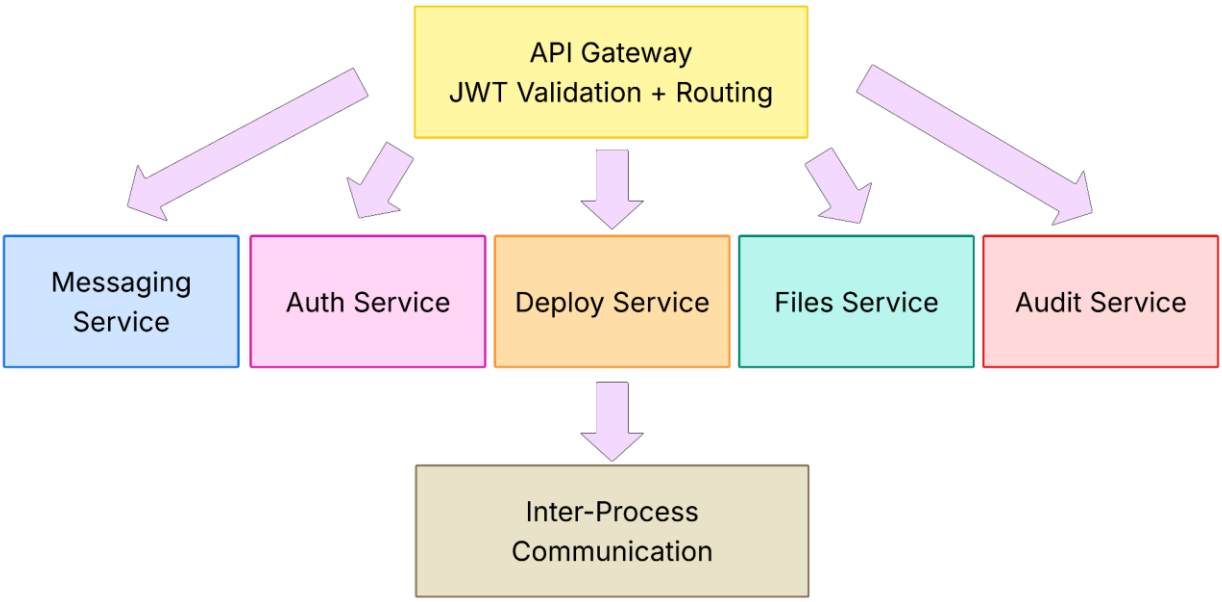


La Révélation Architecturale

Jour 5, Thomas vous convoque en urgence

"J'ai passé la nuit à réfléchir. Nos concurrents échouent parce qu'ils empilent des technologies web qui ne scalent pas sous charge. Nous, allons faire du C++ natif dès le départ. Un seul langage, performance maximale, contrôle total."

Il dessine l'architecture complète au tableau :



Communication Inter-Services C++

Thomas explique la communication entre services :

"Pas de HTTP entre services, c'est trop lent. Named pipes sous Windows, domain sockets sous Linux. Shared memory pour les gros transferts de données. Performance native."

Exemple d'Interaction Complexe

Utilisateur envoie fichier dans chat :

1. Qt Interface → Messaging Service (nouveau message)

2. Messaging → Auth Service (vérification permissions)

3. Messaging → Files Service (upload fichier)

4. Files → Audit Service (event "file_uploaded")

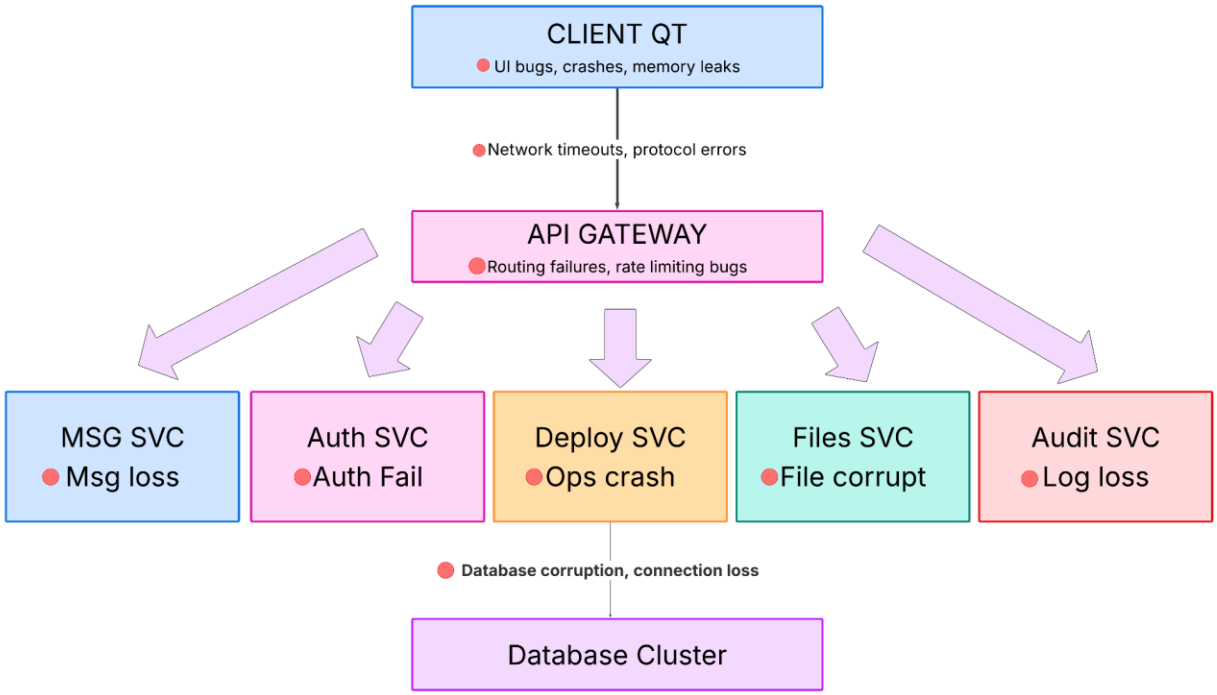
5. Audit → Qt Interface (notification admin si gros fichier)

Stratégie de tests généralisée

Thomas efface le tableau et écrit en gros : "MICROSERVICES = COMPLEXITÉ × 5 = TESTS × 10"

"Écoutez-moi bien. Tester un monolithe, c'est compliqué. Tester 5 microservices qui communiquent entre eux, c'est un cauchemar. Mais MSF nous fait confiance pour protéger des vies humaines. Chaque service, chaque interaction, chaque scénario doit être testé à 100%."

Il dessine l'architecture complète avec les points de défaillance possibles.



Tests Contextualisés MSF (Obligatoire)

Auth Service - Scénarios Critiques

- Mission Afghanistan : Login satellite instable
- Évacuation urgence : Révocation immédiate accès

Files Service - Données Médicales

- Scanner urgence : Upload 500MB <3min
- Backup terrain : Sauvegarde sans internet

Messaging Service - Communication Médicale

- Urgence chirurgicale : Messages priorité absolue
- Évacuation d'urgence : Message immédiat à toutes les équipes

Module 5 : Déploiement CI/CD avec Docker

Thomas se tourne vers l'écran et ouvre GitLab

> *"Dernière étape cruciale : automatiser le déploiement. MSF ne peut pas se permettre des déploiements manuels avec des scp et des prières. Chaque commit doit déclencher une pipeline complète : build, test, deploy automatique."*

Il vous montre un schéma simple au tableau.

> *"L'objectif est simple : git push → 5 minutes plus tard → SecureCloud est déployé et opérationnel. Docker + CI/CD, c'est tout. Mais fait correctement."*

Problèmes du Déploiement Manuel MSF

- ***Temps*** : *3 jours pour déployer sur tous les serveurs*
- ***Erreurs*** : *40% des déploiements échouent*
- ***Inconsistance*** : *Versions différentes selon les pays*
- ***Rollback*** : *Impossible de revenir en arrière rapidement*
- ***Documentation*** : *Procédures perdues/obsolètes*

Story Utilisateur Principale

En tant que *développeur SecureCloud*
Je veux *déployer automatiquement après une action définie*
Afin de *livrer facilement et sans erreurs*

Étapes de la Pipeline

- **Build Stage** : Compilation des 5 microservices C++
- **Test Stage** : Tests unitaires + intégration automatiques
- **Package Stage** : Création des images Docker optimisées
- **Deploy Stage** : Déploiement sur l'environnement cible
- **Verify Stage** : Tests de santé post-déploiement

Orchestration avec Docker Compose

Thomas ouvre le docker-compose.yml

> *"Docker Compose orchestre nos 5 microservices + base de données. Un seul docker-compose up et tout SecureCloud démarre."*

Services Orchestrés

- **auth-service** : Microservice authentification
- **messaging-service** : Communication temps réel
- **files-service** : Partage de fichiers sécurisé
- **audit-service** : Logs et conformité
- **deploy-service** : Gestion infrastructure
- **postgresql** : Base de données

Conclusion

Thomas ferme son laptop et se tourne vers vous

> *"Voilà votre mission : construire SecureCloud en 3 mois. Cinq modules, architecture microservices C++, déploiement automatisé. MSF compte sur vous."*

Il pointe la photo d'équipe MSF au mur.

> *"Ce n'est pas qu'un projet d'école. Votre code pourrait un jour sauver des vies. Chaque ligne de C++ que vous écrirez sera au service de l'humanitaire."*

Thomas vous serre la main

> *"Maintenant, au travail. Le monde nous attend."*

Bon développement, futurs ingénieurs SecureCloud !

Compétences visées

- ➔ Installer et configurer son environnement de travail en fonction du projet.
- ➔ Développer des interfaces utilisateur
- ➔ Développer des composants métier
- ➔ Contribuer à la gestion d'un projet informatique
- ➔ Analyser les besoins et maquetter une application
- ➔ Définir l'architecture logicielle d'une application
- ➔ Préparer et exécuter les plans de tests d'une application

Rendu

Votre travail est évalué en présentation avec un support et une revue de code.
Le slide doit être composé de :

- ➔ De l'organisation de votre équipe
- ➔ De vos problèmes rencontrés ainsi que les solutions apportées
- ➔ La démonstration de votre programme

Le projet est à rendre sur **https://github.com/prenom-nom/wizzMania**

Base de connaissances

- ➔ Librairie Winsock de windows ou en français
- ➔ Avec C++
- ➔ GoogleTest User's Guide
- ➔ Qt Documentation
- ➔ Microservices : définition et architecture | Talend
- ➔ REST API Tutorial

- ➔ WebSockets - Les API Web | MDN
- ➔ Libsodium documentation
- ➔ MSF